

Lab 4 — Introduction to Go

Language Basics · Network Tools · Port Scanner Challenge

Table of contents

1	Introduction	2
2	Setting Up Go	3
2.1	Installation	3
2.2	Your First Go Program	3
3	Go Language Basics	4
3.1	Packages and Imports	4
3.2	Variables	5
3.3	Functions	5
3.4	Error Handling	6
3.5	Loops	6
3.6	Conditionals	6
3.7	Command-Line Arguments	7
3.8	String Formatting	8
4	Networking in Go	8
4.1	Opening a TCP Connection	8
4.2	Setting a Timeout	9
4.3	Goroutines — Concurrent Scanning	10
5	The Challenge — Build a Port Scanner	11
5.1	Overview	11
5.2	Milestone 1 — Single Port Check	11
5.3	Milestone 2 — Scan a Range Sequentially	12
5.4	Milestone 3 — Accept Command-Line Arguments	12
5.5	Milestone 4 — Make It Concurrent	13
5.6	Milestone 5 — Label Well-Known Ports (<i>Stretch Goal</i>)	14
5.7	Milestone 6 — Scan Another VM (<i>Stretch Goal</i>)	14

6 Putting It All Together — Reference Implementation Outline	15
7 Summary	16

1 Introduction

Go (also called Golang) is a compiled, statically-typed programming language created at Google in 2009. It was designed with two goals that make it particularly well-suited for cybersecurity tooling: **simplicity** and **concurrency**. Go programs compile to a single self-contained binary with no runtime dependencies, which means a tool you write on your laptop can be dropped onto a target machine and run immediately — no Python interpreter, no package manager, nothing else required.

A significant portion of modern security tooling is written in Go, including:

Tool	What It Does
nmap alternative naabu	Fast port scanner
Gobuster	Directory and DNS brute-forcer
Nuclei	Vulnerability scanner
Metasploit modules	Some exploit modules
Caddy	Web server used in red team infrastructure

By the end of this lab you will have written your own port scanner in Go — a foundational tool in any attacker’s or defender’s toolkit.

i Why Go for Security Tools?

Three properties make Go a natural fit for network security tools:

- **Speed:** Go compiles to native machine code, so it runs much faster than Python for CPU- or I/O-bound tasks like scanning thousands of ports.
- **Concurrency:** Go has first-class support for goroutines — lightweight threads that make it trivial to scan hundreds of ports simultaneously.
- **Portability:** A compiled Go binary runs on any machine with the same architecture, no dependencies needed. This is invaluable when deploying tools during a competition or engagement.

2 Setting Up Go

2.1 Installation

Install Go on your VM with the following command:

```
sudo apt-get install golang-go
```

Verify the installation succeeded:

```
go version
```

You should see output like `go version go1.XX.x linux/amd64`. The exact version depends on your distro's package repository, and it's normal for `apt`'s version to lag a release or two behind the latest upstream release — that won't affect anything in this lab.

2.2 Your First Go Program

Create a new file called `hello.go`:

```
nano hello.go
```

Type the following:

```
package main

import "fmt"

func main() {
    fmt.Println("Hello, World!")
}
```

Run it directly (compile + execute in one step):

```
go run hello.go
```

Or compile it to a binary first, then run the binary:

```
go build hello.go
./hello
```

go run vs. go build

- `go run` compiles and runs in one step. Good for development and quick tests.
- `go build` produces a standalone binary. Good for tools you want to deploy or share — the resulting file has no dependencies and can be copied anywhere.

3 Go Language Basics

If you already know Python, C, or Java, most of Go will feel familiar. This section highlights the parts that are different or worth paying attention to before the challenge.

3.1 Packages and Imports

Every Go file starts by declaring which **package** it belongs to. Executable programs always use package `main`. The `import` block brings in standard library or third-party packages.

```
package main

import (
    "fmt"        // formatted I/O (like print)
    "net"        // networking (TCP/UDP connections, DNS)
    "os"         // OS interaction (args, exit)
)
```

Unused Imports are a Compile Error

Go will refuse to compile if you import a package and never use it. This is intentional — it keeps codebases clean. If you add an import while experimenting, make sure you use it or remove it before running.

3.2 Variables

Go is statically typed, but the `:=` operator lets you declare and assign in one line without writing the type explicitly — Go infers it.

```
// Explicit type declaration
var port int = 80

// Short declaration (type inferred) - most common style
host := "localhost"
open := true

// Multiple variables at once
var (
    minPort = 1
    maxPort = 1024
)
```

3.3 Functions

```
// A function that takes two ints and returns a string
func greet(name string, times int) string {
    result := ""
    for i := 0; i < times; i++ {
        result += "Hello, " + name + "\n"
    }
    return result
}
```

Go functions can return **multiple values** — this is commonly used to return a result alongside an error:

```
func divide(a, b float64) (float64, error) {
    if b == 0 {
        return 0, fmt.Errorf("cannot divide by zero")
    }
    return a / b, nil
}
```

3.4 Error Handling

Go does not have exceptions. Instead, functions return an **error** value as their last return value. The convention is to check it immediately:

```
result, err := divide(10, 0)
if err != nil {
    fmt.Println("Error:", err)
} else {
    fmt.Println("Result:", result)
}
```

You will see this pattern constantly in Go — especially when opening network connections, where many things can go wrong.

3.5 Loops

Go has only one loop keyword: **for**. It covers all three common loop styles:

```
// Classic C-style loop
for i := 0; i < 10; i++ {
    fmt.Println(i)
}

// While-style loop
count := 0
for count < 5 {
    count++
}

// Infinite loop (useful for servers)
for {
    // runs forever until a break or return
}
```

3.6 Conditionals

```
port := 22

if port == 22 {
    fmt.Println("SSH")
} else if port == 80 {
    fmt.Println("HTTP")
} else {
    fmt.Println("Unknown")
}
```

Go also has a `switch` statement that is cleaner for many comparisons:

```
switch port {
case 21:
    fmt.Println("FTP")
case 22:
    fmt.Println("SSH")
case 80:
    fmt.Println("HTTP")
case 3306:
    fmt.Println("MySQL")
default:
    fmt.Println("Unknown service")
}
```

3.7 Command-Line Arguments

The `os.Args` slice holds command-line arguments. `os.Args[0]` is always the program name itself; your actual arguments start at index 1.

```
package main

import (
    "fmt"
    "os"
)

func main() {
    if len(os.Args) < 2 {
        fmt.Println("Usage: ./program <name>")
        os.Exit(1)
    }
}
```

```
}  
name := os.Args[1]  
fmt.Println("Hello,", name)  
}
```

```
go run hello.go Alice  
# Output: Hello, Alice
```

3.8 String Formatting

`fmt.Sprintf` builds a formatted string (like Python's f-strings or C's `sprintf`):

```
host := "192.168.1.1"  
port := 80  
address := fmt.Sprintf("%s:%d", host, port)  
// address is now "192.168.1.1:80"
```

Common format verbs:

Verb	Meaning
%s	String
%d	Integer
%f	Float
%v	Any value (Go picks the format)
%t	Boolean

4 Networking in Go

4.1 Opening a TCP Connection

The `net` package provides everything needed for TCP and UDP networking. To attempt a connection to a host and port, use `net.Dial`:

```

package main

import (
    "fmt"
    "net"
)

func main() {
    // Attempt to connect to port 80 on localhost
    conn, err := net.Dial("tcp", "localhost:80")
    if err != nil {
        fmt.Println("Port closed or unreachable:", err)
        return
    }
    defer conn.Close()
    fmt.Println("Port 80 is open!")
}

```

`defer conn.Close()` is a Go idiom — it schedules the connection to be closed when the surrounding function returns, no matter how it exits. Think of it as a guaranteed cleanup step.

i What Does a TCP Connection Attempt Tell You?

When you call `net.Dial("tcp", host+":port")`:

- If the connection **succeeds**, a service is actively listening on that port.
- If it **fails with “connection refused”**, the port is closed — nothing is listening.
- If it **fails with a timeout**, the port may be filtered by a firewall.

This is exactly the logic that port scanners like **nmap** use at their core.

4.2 Setting a Timeout

Without a timeout, a connection attempt to a filtered port can hang for 30+ seconds. Always set a timeout when scanning:

```

import (
    "net"
    "time"
)

```

```
conn, err := net.DialTimeout("tcp", "localhost:9999", 1*time.Second)
```

`1*time.Second` sets a one-second deadline. If the connection is not established within that time, `err` will be non-nil and you move on.

4.3 Goroutines — Concurrent Scanning

Scanning ports one at a time is slow. Go's **goroutines** let you scan many ports simultaneously. A goroutine is started with the `go` keyword:

```
go someFunction() // runs someFunction concurrently
```

To wait for multiple goroutines to finish, use a `sync.WaitGroup`:

```
package main

import (
    "fmt"
    "sync"
)

func sayHello(id int, wg *sync.WaitGroup) {
    defer wg.Done() // signal this goroutine is done when the function returns
    fmt.Println("Hello from goroutine", id)
}

func main() {
    var wg sync.WaitGroup

    for i := 0; i < 5; i++ {
        wg.Add(1) // tell the WaitGroup to expect one more goroutine
        go sayHello(i, &wg)
    }

    wg.Wait() // block here until all goroutines call wg.Done()
    fmt.Println("All done.")
}
```

💡 Think of a `WaitGroup` as a Counter

`wg.Add(1)` increments a counter. `wg.Done()` decrements it. `wg.Wait()` blocks until the counter reaches zero. Simple but powerful.

5 The Challenge — Build a Port Scanner

5.1 Overview

Working individually or in groups of up to five, your task is to build a concurrent TCP port scanner in Go. Your scanner should:

1. Accept a **target host** and a **port range** as command-line arguments.
2. Attempt a TCP connection to each port in the range.
3. Print each port that is **open**, along with the service name if it is a well-known port.
4. Run scans **concurrently** (using goroutines) so that scanning 1–1024 completes in a few seconds rather than minutes.

5.2 Milestone 1 — Single Port Check

Before worrying about ranges or concurrency, write a program that checks a single hardcoded port on `localhost` and prints whether it is open or closed.

You will need:

- `net.DialTimeout` to attempt the connection
- An `if err != nil` check to determine the result
- `fmt.Println` to report the outcome

Expected output (with Apache running from Lab 3):

Port 80: OPEN

Expected output (for a closed port):

Port 9999: closed

💡 Test Against the Services from Lab 3

You already have Apache (80), FTP (21), SSH (22), and MySQL (3306) running on your VM from Lab 3. Use those as known-open ports to test your scanner as you build it.

5.3 Milestone 2 — Scan a Range Sequentially

Extend your program to loop over a range of ports (e.g. 1–1024) and check each one in turn. At this point, run them sequentially — no goroutines yet.

You will need:

- A for loop from `minPort` to `maxPort`
- `fmt.Sprintf` to build the address string (e.g. "localhost:80")
- Only print ports that are open (suppress closed ports to keep output readable)

Expected output:

```
Port 21:    OPEN
Port 22:    OPEN
Port 80:    OPEN
Port 3306:  OPEN
```

5.4 Milestone 3 — Accept Command-Line Arguments

Hardcoded values are fine for testing but useless as a real tool. Update your program to accept the target host, minimum port, and maximum port from the command line.

```
go run scanner.go localhost 1 1024
```

You will need:

- `os.Args` to read arguments
- `strconv.Atoi` to convert the port strings to integers

i Converting Strings to Integers

Command-line arguments are always strings. Use `strconv.Atoi` to convert them:

```
import "strconv"

portNum, err := strconv.Atoi("8080")
if err != nil {
    fmt.Println("That's not a valid port number")
    os.Exit(1)
}
```

5.5 Milestone 4 — Make It Concurrent

Sequential scanning of 1024 ports can take over a minute with a 1-second timeout per port. Add goroutines so all ports are scanned in parallel.

Replace your sequential loop with a goroutine-per-port pattern using a `sync.WaitGroup`.

Beware of the Loop Variable Gotcha

A classic Go mistake is closing over the loop variable in a goroutine:

```
// BUG: all goroutines may see the same value of `port`
for port := 1; port <= 1024; port++ {
    go func() {
        scan(port)    // port may have already changed!
    }()
}

// FIX: pass port as an argument to the goroutine function
for port := 1; port <= 1024; port++ {
    wg.Add(1)
    go func(p int) {
        defer wg.Done()
        scan(p)    // p is a copy, safe to use
    }(port)
}
```

After adding concurrency, time your scanner:

```
time go run scanner.go localhost 1 1024
```

You should see the scan complete in a few seconds rather than minutes.

5.6 Milestone 5 — Label Well-Known Ports (*Stretch Goal*)

Update your output so that well-known ports are labelled with their service name.

```
Port 21:    OPEN  (FTP)
Port 22:    OPEN  (SSH)
Port 80:    OPEN  (HTTP)
Port 3306:  OPEN  (MySQL)
```

Use a `map` to store the port-to-service name lookup:

```
services := map[int]string{
    21:  "FTP",
    22:  "SSH",
    23:  "Telnet",
    25:  "SMTP",
    53:  "DNS",
    80:  "HTTP",
    443: "HTTPS",
    3306: "MySQL",
    5432: "PostgreSQL",
    8080: "HTTP-Alt",
}

name, known := services[port]
if !known {
    name = ""
}
```

5.7 Milestone 6 — Scan Another VM (*Stretch Goal*)

Point your scanner at a classmate's VM instead of localhost.

```
go run scanner.go 10.4.5.7 1 1024
```

Compare your results. Do they have the same services running? Are there any unexpected open ports?

! Ethical Use

Only scan machines you have explicit permission to scan — your own VM or a classmate's VM in the lab environment. Running a port scanner against systems you do not own is illegal and against Louisiana Tech's acceptable use policy.

6 Putting It All Together — Reference Implementation Outline

The structure below is a suggested skeleton. Fill in the blanks as you work through the milestones. Do not copy-paste — write it yourself and understand each line.

```
package main

import (
    "fmt"
    "net"
    "os"
    "strconv"
    "sync"
    "time"
)

// checkPort attempts a TCP connection to host:port.
// It returns true if the port is open, false otherwise.
func checkPort(host string, port int) bool {
    address := fmt.Sprintf("%s:%d", host, port)
    conn, err := net.DialTimeout("tcp", address, 1*time.Second)
    if err != nil {
        return false
    }
    conn.Close()
    return true
}

func main() {
    // TODO Milestone 3: Read host, minPort, maxPort from os.Args
    // TODO Milestone 3: Convert port strings to ints with strconv.Atoi
    // TODO Milestone 4: Create a sync.WaitGroup
```

```
// TODO Milestone 2/4: Loop over ports, call checkPort, print open ones
// TODO Milestone 5: Label well-known ports using a map
// TODO Milestone 4: wg.Wait() before exiting
}
```

7 Summary

Concept	Go Feature Used
TCP connection attempt	<code>net.DialTimeout</code>
Concurrent scanning	<code>goroutines</code> + <code>sync.WaitGroup</code>
Command-line input	<code>os.Args</code> + <code>strconv.Atoi</code>
String formatting	<code>fmt.Sprintf</code>
Port name lookup	<code>map[int]string</code>
Error handling	<code>if err != nil</code>

Going Further

- **UDP scanning:** TCP connect scanning does not work for UDP services (DNS on port 53, DHCP on 67/68). Research how UDP scanning differs and why it is harder.
- **Banner grabbing:** After a successful TCP connection, many services immediately send a banner (e.g. SSH version strings, FTP welcome messages). Try reading the first few bytes from the connection with `bufio.NewReader` and printing them.
- **Output to a file:** Use Go's `os.Create` and `fmt.Fprintf` to write results to a `.txt` report file instead of just printing to the terminal.
- **Compare with nmap:** Run `nmap -p 1-1024 localhost` and compare its output to your scanner. Where do they agree? Where do they differ, and why?